

Enterprise Smart Library Management Platform

Infrastructure Automation using Terraform and Ansible on Amazon Web Services (AWS) — Unit 4 Comprehensive Engineering Architecture Manual

Curriculum Unit	Target Cloud Platform	IaC & Automation Tools	Manual Status
UNIT 4: DevOps Automation	Amazon Web Services (AWS)	Terraform (v1.7+) & Ansible (v2.16+)	v1.0.0 Production Architecture

1. Executive Summary & Infrastructure Automation Imperative

In the preceding units of the Careertiq curriculum, software engineering teams successfully transformed an enterprise business concept into an Agile product roadmap, engineered a modular Python/Flask microservice, established continuous automated testing via Pytest, and containerized the runtime using Docker and Kubernetes. However, in enterprise production, writing robust application code is only half the software lifecycle equation.

Before any application can serve customer traffic, organizations must provision and maintain complex supporting infrastructure: virtual networks, subnets, routing tables, security firewalls, cloud compute instances, storage volumes, OS-level dependencies, runtime daemons, reverse proxies, and backup stores. Historically, infrastructure operations relied on manual configuration—commonly referred to in industry as "**ClickOps**" or manual server administration via SSH.

Manual administration introduces critical points of failure: protracted provisioning times, unrecorded human errors, security misconfigurations, and severe **configuration drift** between staging and production environments. Unit 4 introduces **Infrastructure as Code (IaC)** and **Configuration Management** to automate the complete infrastructure lifecycle. Using the **Enterprise Smart Library Management Platform** as the reference case study, this document demonstrates how modern software organizations declare cloud infrastructure via **Terraform** and automate server configuration via **Ansible** on AWS.

CORE ARCHITECTURAL OBJECTIVE

Eliminate manual server setup through declarative Infrastructure as Code (Terraform) and idempotent Configuration Management (Ansible). By codifying the infrastructure lifecycle, organizations achieve 100% environment reproducibility, zero configuration drift, immutable audit trails, and automated zero-touch server provisioning.

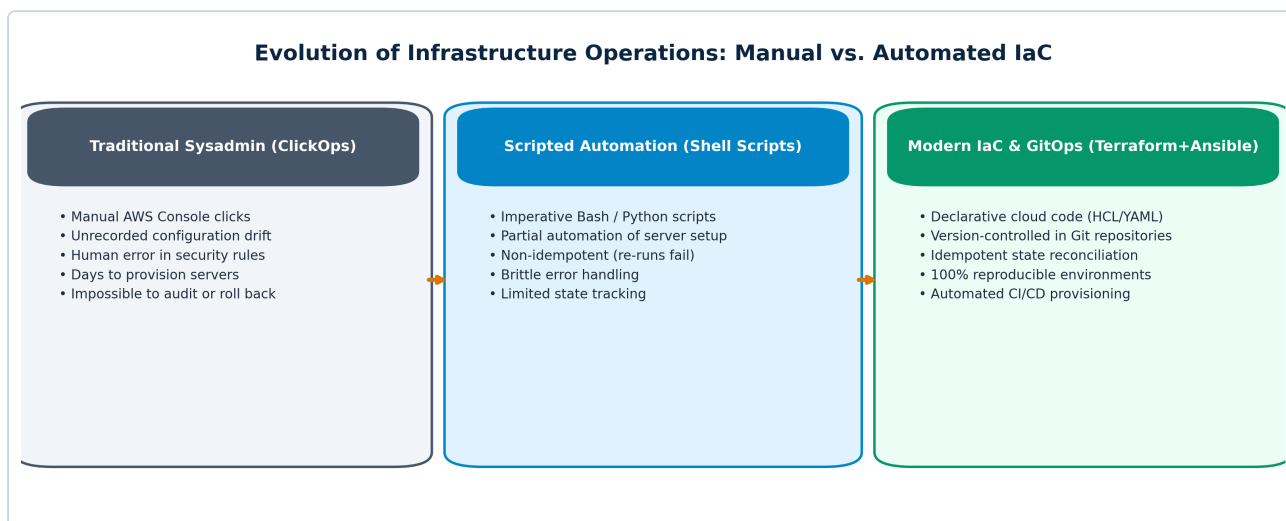


Figure 1.1: Evolution of Software Infrastructure Management from Manual Runbooks to Cloud-Native IaC

2. Comprehensive Infrastructure Requirements Analysis

Infrastructure planning must always precede provisioning. To ensure optimal performance, security isolation, and cost governance, engineers must analyze the exact computational and networking prerequisites of the application components.

2.1 Application Components vs. Cloud Infrastructure Mapping

The Enterprise Smart Library Management Platform consists of distinct functional layers. Table 2.1 maps each application layer to its required AWS infrastructure primitive.

CAREERTIQ

Application Component	Runtime Function	Target AWS Infrastructure	Operational Specifications
Python Web API	Flask 3.0 REST Endpoints, Gunicorn WSGI Server	AWS EC2 Instance (Compute)	Instance type: t2.micro (1 vCPU, 1 GiB RAM), Ubuntu 22.04 LTS, EBS 20GB gp3 root volume.
Database Storage	Library book records, borrower accounts, loan transactions	Persistent Storage / Local DB	Thread-safe SQLite store with mounted volume persistence; exportable to AWS RDS PostgreSQL for scale.
Web Reverse Proxy	SSL Termination, HTTP request buffering, static proxy	Nginx Web Server	Listens on public Port 80; reverse proxies internal requests to Gunicorn Docker container on Port 5000.
Container Runtime	Executes packaged Docker container image from Unit 3	Docker Engine & containerd	Container restart policy always; network port mapping 5000:5000; isolated cgroups & namespaces.
Virtual Network	Network boundary, CIDR isolation, packet routing	AWS Virtual Private Cloud (VPC)	Dedicated VPC 10.0.0.0/16; Public Subnet 10.0.1.0/24; Internet Gateway (IGW); Route Table.
Network Security	Ingress traffic filtering, port isolation	AWS Security Group (Stateful)	Restricted Ingress: Port 22 (SSH Management), Port 80 (HTTP), Port 5000 (Direct API). Unrestricted Egress.
Backup Storage	Nightly database dumps, audit logs, Terraform state	AWS S3 Object Bucket	Private encrypted bucket (AES-256); S3 Versioning enabled; lifecycle policies for disaster recovery.

2.2 Multi-Tier Environment Variations

Infrastructure requirements differ across lifecycle environments. The automation architecture must accommodate:

- **Development Environment:** Optimized for rapid developer feedback and minimal cost. Utilizes single Availability Zone (AZ) deployment, ephemeral storage, and relaxed security groups for local testing.
- **Testing & Staging Environment:** 100% replica of production. Executes automated CI/CD integration tests, load tests, and security vulnerability scans.
- **Production Environment:** High availability, multi-AZ resilience, Elastic IP binding, encrypted S3 backups, strict least-privilege IAM roles, and automated alerting.

3. Cloud Architecture Design on Amazon Web Services (AWS)

Modern cloud architectures separate public traffic routing from internal application runtime. Figure 3.1 illustrates the complete enterprise cloud deployment topology designed for the Library Management Platform.

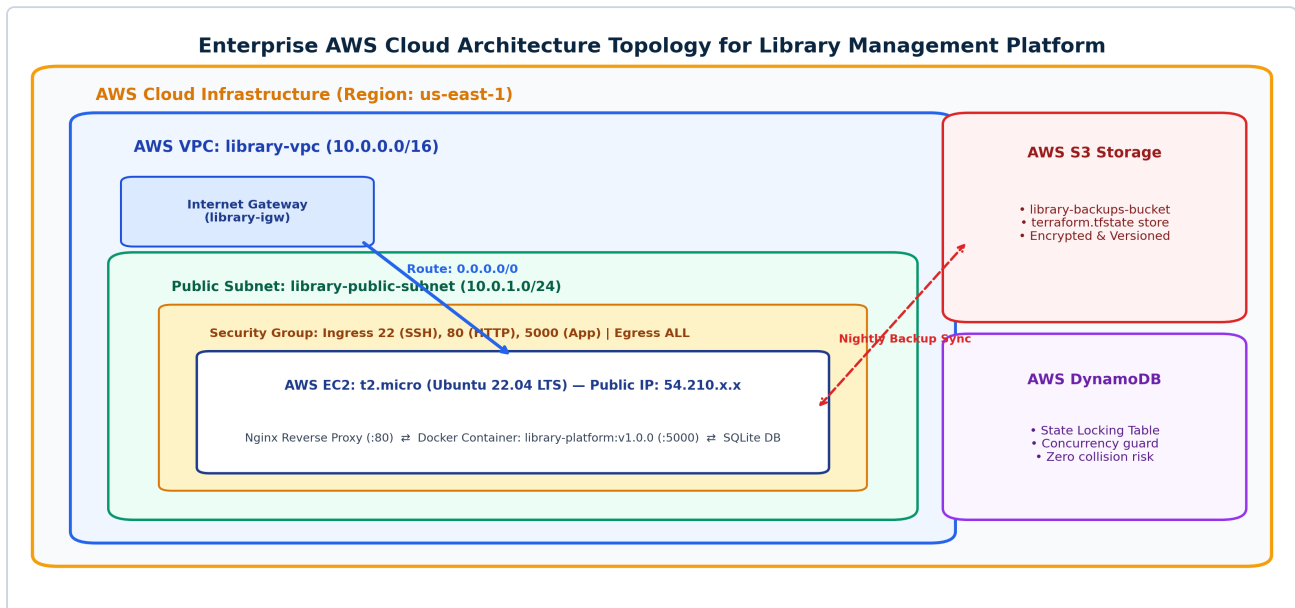


Figure 3.1: Enterprise AWS Cloud Architecture Topology for Library Management Platform

3.1 Architectural Layer Breakdown

- **Network Isolation Layer:** The platform resides within a custom AWS Virtual Private Cloud (VPC) assigned the CIDR block 10.0.0.0/16. An Internet Gateway (IGW) is attached to the VPC to enable bidirectional internet communication. A Public Subnet (10.0.1.0/24) hosts the compute instance, with default routes (0.0.0.0/0) directed to the IGW.
- **Security & Ingress Layer:** A stateful AWS Security Group acts as a virtual firewall. Ingress traffic is restricted to Port 22 (SSH administrator access), Port 80 (Nginx HTTP web traffic), and Port 5000 (direct application access). Outbound egress is open to allow APT updates and Docker image pulls.
- **Compute & Runtime Layer:** An AWS EC2 instance running Ubuntu 22.04 LTS executes Docker Engine. Inside Docker, the Enterprise Smart Library container runs isolated, exposing port 5000. Nginx functions as a high-performance reverse proxy on Port 80, buffering client requests and forwarding traffic to the container.
- **Persistent Storage & State Layer:** An AWS S3 bucket maintains versioned, encrypted backups of library transactions. Concurrently, S3 and AWS DynamoDB provide remote state storage and distributed locking for Terraform execution.

4. Infrastructure as Code (IaC) with HashiCorp Terraform

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable definition files rather than physical hardware configuration or interactive GUI tools. HashiCorp Terraform is the industry-standard cloud-agnostic IaC engine.

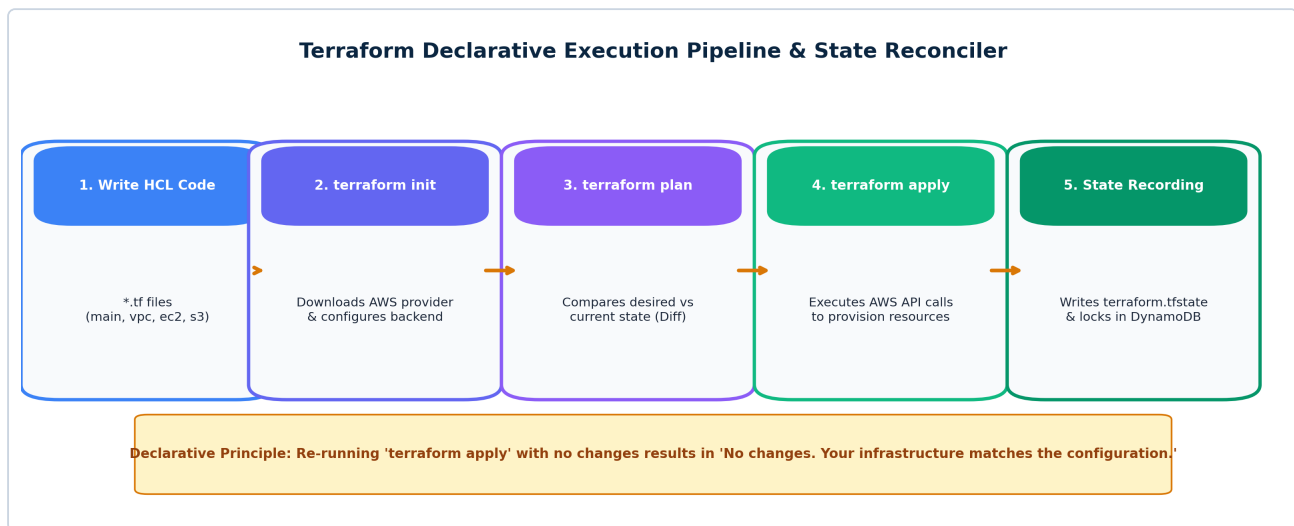


Figure 4.1: Terraform Declarative Execution Pipeline and State Reconciliation Engine

4.1 Core Concepts and Principles of Terraform

- **Declarative Syntax (HCL):** HashiCorp Configuration Language allows engineers to define the *desired end state* of infrastructure. The engineer specifies *what* resources should exist, and Terraform calculates *how* to achieve that state.
- **Terraform Providers:** Plugins that interact with cloud platforms via official APIs. For AWS, the hashicorp/aws provider translates HCL declarations into AWS REST API requests.
- **State Management (terraform.tfstate):** Terraform records resource metadata, dependencies, and IDs in a state file. This state file serves as the single source of truth mapping declared resources to real-world cloud objects.
- **State Locking & Concurrency:** In team environments, concurrent Terraform executions can corrupt the state file. Terraform integrates with AWS DynamoDB to enforce distributed mutex locking during operations.

4.2 The Five-Stage Terraform Command Lifecycle

Terraform Command	Internal Execution Mechanism	Operational Outcome
terraform init	Initializes the working directory, downloads required provider plugins (hashicorp/aws), and connects to remote state backends.	Creates .terraform/ directory and locks dependencies in .terraform.lock.hcl.
terraform validate	Parses configuration files syntactically and semantically without making remote API calls.	Confirms HCL syntax correctness, valid variable types, and attribute presence.
terraform plan	Queries cloud APIs to inspect current state, compares it with local HCL declarations, and constructs an execution graph.	Generates an execution plan detailing resources to be created (+), modified (~), or destroyed (-).
terraform apply	Executes the dependency graph in parallel, calling AWS APIs to provision resources in topological order.	Provisions VPC, Subnet, Security Group, S3, and EC2, then updates terraform.tfstate.
terraform destroy	Reverses the dependency graph and systematically deprovisions all managed cloud resources.	Terminates EC2, deletes S3 buckets, removes VPC, and frees cloud infrastructure costs.

5. Configuration Management with Ansible

While Terraform provisions cloud fabric (hardware, networking, instances), it is not designed for granular operating system and software configuration. That task belongs to **Configuration Management** tools such as Ansible.

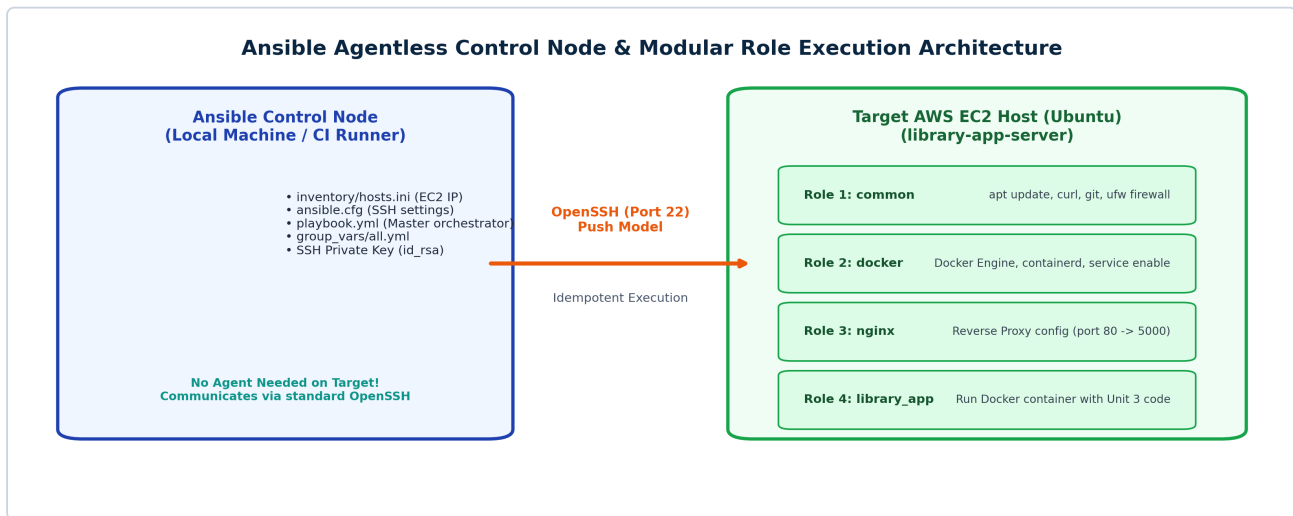


Figure 5.1: Ansible Agentless Control Node Architecture & Modular Role Execution

5.1 Why Ansible? Key Architectural Advantages

- **Agentless Architecture:** Unlike legacy tools (Puppet, Chef) that require proprietary daemons running on target servers, Ansible operates over standard OpenSSH. Target servers require only a Python interpreter.
- **Push-Based Model:** The Ansible Control Node pushes configurations directly to target hosts on demand, eliminating client pull delays and central server load.
- **Idempotency:** An operation is idempotent if executing it multiple times produces the exact same outcome without unintended side effects. If a package (e.g., Docker) is already installed, Ansible reports ok and skips reinstallation.
- **Human-Readable YAML:** Playbooks and roles are declared in straightforward YAML syntax, facilitating code reviews and cross-team collaboration.

5.2 Modular Ansible Roles Hierarchy

Enterprise configurations are structured into reusable **Roles**. For the Smart Library Platform:

- **roles/common:** Updates APT repository caches, installs system utilities (curl, git, jq), and configures UFW firewall.
- **roles/docker:** Adds official Docker GPG keys, installs Docker CE and containerd, enables systemd daemons, and adds non-root users to the docker security group.
- **roles/nginx:** Installs Nginx, generates dynamic reverse proxy configurations via Jinja2 templates, and proxies Port 80 to container Port 5000.
- **roles/library_app:** Deploys the containerized application artifact from Unit 3, mounts persistent database directories, launches the container with restart policies, and verifies health endpoints.

6. The Two-Tier DevOps Orchestration Handshake

DevOps excellence requires combining Terraform and Ansible into a unified, automated delivery continuum. Figure 6.1 illustrates the two-tier orchestration handshake.

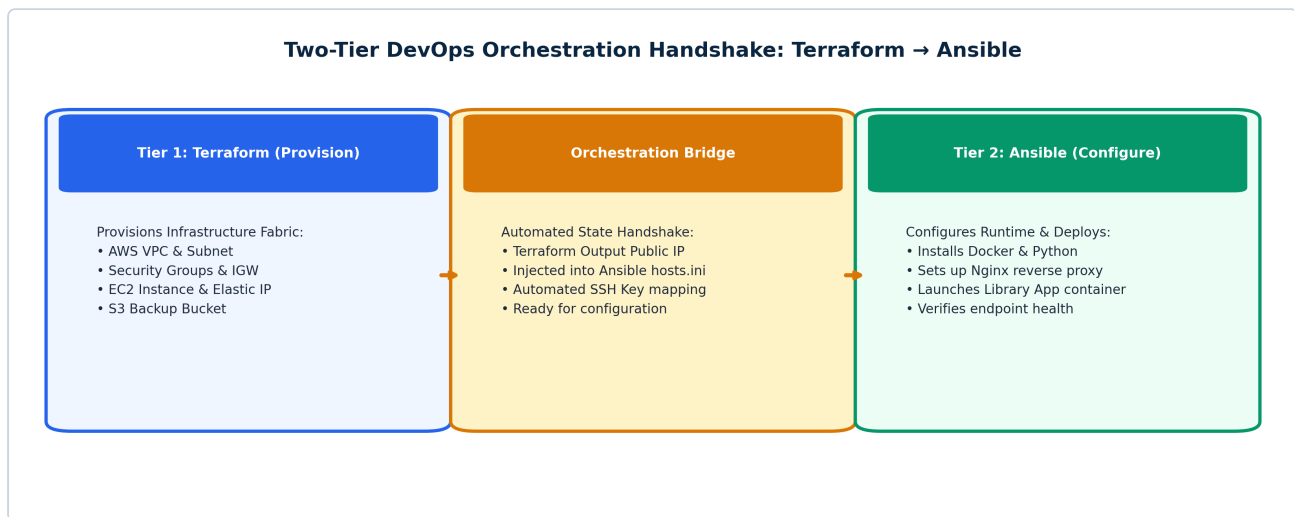


Figure 6.1: Two-Tier DevOps Orchestration Handshake: Terraform Provisions, Ansible Configures

6.1 The Orchestration Handoff Mechanism

The central engineering challenge is connecting the dynamic outputs of Terraform (such as randomly assigned AWS public IP addresses) to Ansible's static inventory. Modern DevOps solves this via automated templating:

Automated Handshake Sequence:

1. Terraform provisions EC2 instance and outputs: `app_server_public_ip = "54.210.88.14"`
2. CI pipeline or wrapper script queries: `terraform output -raw app_server_public_ip`
3. Output dynamically populates Ansible inventory:


```
[library_servers]
  library-host ansible_host=54.210.88.14 ansible_user=ubuntu
```
4. Ansible executes playbook against newly created host via SSH.

7. Environment Parity & Multi-Tier Promotion Pipeline

Environment inconsistency is a primary driver of software deployment failure. If development, testing, and production servers differ in OS patch levels, Python minor versions, or firewall configurations, software that passed tests will fail in production.

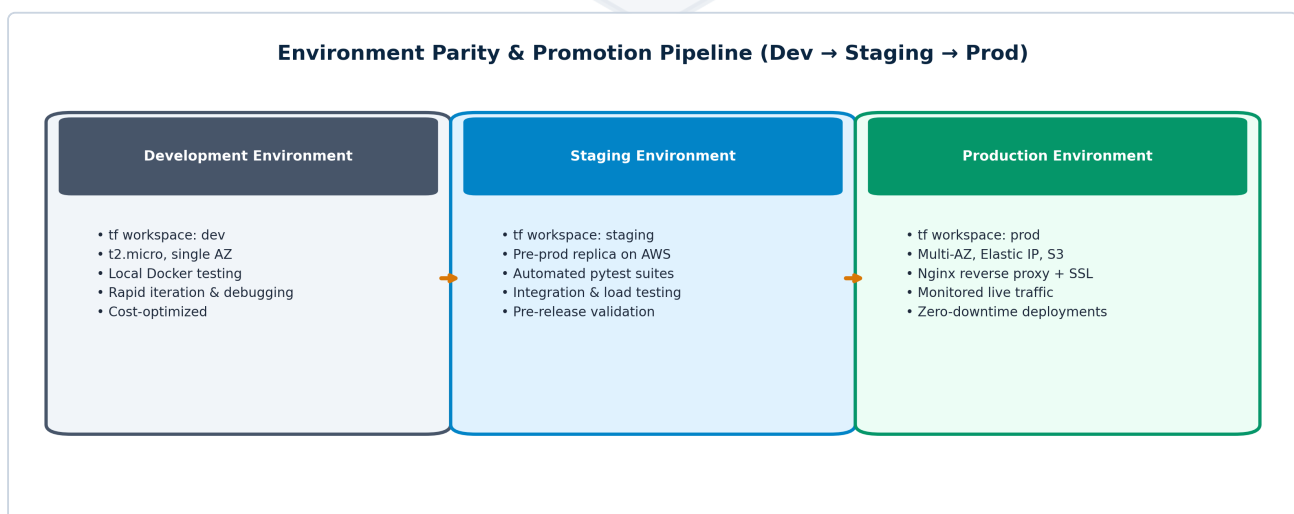


Figure 7.1: Environment Parity and Promotion Pipeline across Development, Staging, and Production

7.1 Achieving 100% Parity via Code

- **Terraform Workspaces:** A single Terraform codebase defines infrastructure across environments by switching workspaces: `terraform workspace select dev` vs. `prod`. Variables dynamically adjust instance types and replica counts while maintaining identical architecture.
- **Ansible Group Variables:** Environment-specific parameters (e.g., database connection strings, logging verbosity) reside in `group_vars/dev.yml` and `group_vars/prod.yml`, while tasks remain completely identical.

8. Infrastructure Validation & Security Hardening

Deploying infrastructure is incomplete without rigorous post-deployment validation. Production systems mandate automated smoke testing across all architectural layers.

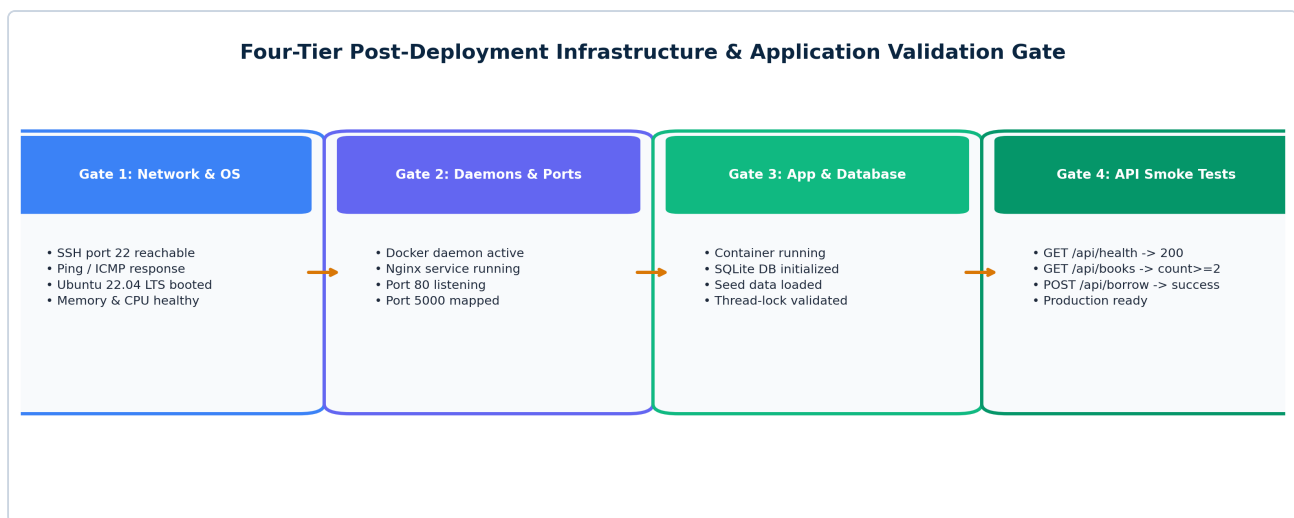


Figure 8.1: Four-Tier Post-Deployment Infrastructure and Application Validation Gate

8.1 The Four Validation Gates

Validation Gate	Verification Objective	Automated Validation Command
Gate 1: Network & OS	Confirms instance reachable via network, SSH port 22 open, and Ubuntu kernel initialized.	<code>nc -zv -w 5 \$EC2_IP 22 and ssh -i key.pem ubuntu@\$EC2_IP uptime</code>
Gate 2: Daemons & Ports	Verifies system services active, Docker daemon running, and Nginx listening on Port 80.	<code>ansible -m systemd -a "name=docker state=started" library_servers</code>
Gate 3: App Container	Ensures Docker container status is Up, internal port 5000 mapped, SQLite database writable.	<code>ssh ubuntu@\$EC2_IP "docker ps --filter name=library-app --format '{{.Status}}'"</code>
Gate 4: API Smoke Tests	Executes HTTP GET and POST requests against REST endpoints to confirm live application health.	<code>curl -s -f http://\$EC2_IP/api/health jq .status (Must return "UP")</code>

9. Student Project Adaptation: Student Record & Academic Platform

Following the trainer's demonstration on the Library Management Platform, students independently automate the cloud infrastructure for their parallel academic project: the **Enterprise Student Record and Academic Management Platform**.

Curriculum Dimension	Trainer Case Study (Library Platform)	Student Implementation (Academic Platform)
Application Domain	Library book circulation, borrowings, inventory	Student records, course enrollment, academic marks
Compute Sizing	AWS EC2 t2.micro (1 vCPU, 1GB RAM)	AWS EC2 t2.micro (1 vCPU, 1GB RAM)
Terraform VPC & SG	VPC: 10.0.0.0/16, SG: 22, 80, 5000	VPC: 10.1.0.0/16, SG: 22, 80, 8080
Ansible Role Focus	Docker container running Library Flask app	Docker container running Student Record Flask app
API Smoke Tests	GET /api/health, GET /api/books, POST /api/borrow	GET /api/health, GET /api/students, POST /api/marks

9.1 Evaluation Rubric & Submission Deliverables

- **1. Terraform Configuration Files (30%):** Clean, modular HCL definitions for VPC, Subnet, Security Group, EC2, and S3. Must execute cleanly without manual intervention.
- **2. Ansible Automation Roles (30%):** Modular roles for system setup, Docker installation, Nginx configuration, and application deployment. Playbooks must demonstrate strict idempotency.
- **3. End-to-End Cloud Deployment (20%):** Successful deployment of the containerized application on AWS EC2, reachable via public IP on Port 80.
- **4. Validation & Technical Report (20%):** Submission of automated smoke test logs, architecture diagrams, and GitHub repository link.

CAREERTIQ